

Parallelism Fundamentals

Satyadhyan Chickerur, Ph.D., MBA(Edu.Mgmt), SMIEEE
Director, Centre for Artificial Intelligence Research,
Prof., School of Computer Science & Engineering,
KLE Technological University,
BVB Campus, Vidyanagar,
Hubballi - 580031, Karnataka, India

SESSION 1 OF 4

Why Distributed — The Math Doesn't Fit Anymore

1 Model size

70B params in fp16 = 140 GB. Twice the memory of an H100-80. And that's just weights — optimizer state and activations add more.

2 Dataset size

LLM pretrain budget: ~15T tokens. At ~3-5B tokens/day on one GPU for a 7B model, that's years — not a viable timeline.

3 Wall-clock pressure

Research iteration needs days, not weeks. Scale compresses 2-week single-GPU experiments down to hours across a cluster.

Pick the reason that matches your problem :
iteration time, model size, both.

Four Axes to Parallelise Along

1 Data Parallelism

Replicate: model. Split: batch. Comm: all-reduce gradients once per step.

Default choice — use when the model fits and you want more throughput.

2 Tensor Parallelism

Replicate: batch. Split: weight tensors (cols/rows). Comm: all-reduce activations INSIDE each layer.

Use when one layer doesn't fit (Megatron-LM style).

3 Pipeline Parallelism

Split: layers. Comm: activations forward, gradients backward.

Use when the model is too deep for one GPU (GPipe / 1F1B).

4 Sharded (Fully Sharded Data Parallel (FSDP) / Zero Redundancy Optimizer,)

Split: weights + gradients + optimizer states. Comm: all-gather weights, reduce-scatter grads.

Use when memory-bound but you still want DDP-like speed.

3D Parallelism — Combine All Three Axes

At frontier scale you combine data + tensor + pipeline at once. Each GPU has a (p, t, d) coordinate; dimensions multiply — $4 \times 8 \times 4 = 128$ GPUs.

p=0 t=0 d=0

p=0 t=0 d=1

p=1 t=0 d=0

p=1 t=0 d=1

p=0 t=1 d=0

p=0 t=1 d=1

p=1 t=1 d=0

p=1 t=1 d=1

data dim (across columns) · tensor dim (rows) · pipeline dim (across planes)

Topology matters: tensor ranks live on the same NVLink (high-frequency comm), pipeline stages span nodes (medium), data replicas span racks (low-frequency).

- Start with DDP.
- Memory problem? FSDP or ZeRO-3.
- One LAYER too big? Tensor parallel.
- Model too DEEP? Pipeline.
- All at once? 3D parallel — pure FSDP handles most cases under 100B.

The Vocabulary of a Distributed Launcher

World size

Total number of processes. One per GPU.
Example: 2 nodes x 4 GPUs = world_size 8.

Rank (global)

0 to world_size-1. Unique per process across the whole job.

Local rank

0 to gpus_per_node-1. Picks which GPU on THIS node to bind to.

Node rank / Process group

Node rank: 0 to nnodes-1, which machine.
Process group: the communicator — default is all ranks.

```
torchrun SETS THESE ENV VARS FOR YOU:  
RANK=5           # global rank in [0, world_size)  
LOCAL_RANK=1     # local rank in [0, gpus_per_node)  
WORLD_SIZE=8     # total processes  
MASTER_ADDR=node0.example.com  MASTER_PORT=29500
```

Communication Backends

NCCL

NVIDIA Collective Communications Library. GPU-to-GPU: NVLink intra-node, RDMA over IB/RoCE inter-node. Topology-aware.
WHEN: default for GPU training — almost always this.

Gloo

Facebook's cross-platform backend. CPU-friendly, portable, no special deps. No GPU acceleration.
WHEN: debugging on CPU only — not production.

MPI

Message Passing Interface, the HPC standard. PyTorch supports it but requires a rebuild; Horovod uses MPI under the hood.
WHEN: HPC clusters where MPI is already the norm.

```
dist.init_process_group(backend="nccl")  
# 99% of the time, pick this.
```

Collective Primitives

Broadcast (1 -> N)

Sync initial weights from rank 0 to all ranks.

Reduce (N -> 1)

Sum gradients to one worker (old style, rarely used directly now).

All-reduce (N -> N)

Gradient sync in DDP. THE primitive — the workhorse of distributed training.

All-gather (N -> N)

FSDP uses it to regather sharded parameters before a forward/backward pass.

Reduce-scatter (N -> N)

First half of ring all-reduce — each rank ends up owning one fully-reduced chunk.

Barrier

Sync point. All workers wait until every rank arrives before proceeding.

Key identity: all-reduce = reduce-scatter then all-gather — the foundation of ring all-reduce, covered in full in Session 3.

Where Each Axis Gets Used in Practice

Model / Scenario	Data parallel	Tensor parallel	Pipeline parallel	Sharded
7B fine-tune, 8 GPUs	Yes (DDP)	No	No	Optional (FSDP if tight)
70B pretrain, 1 node (8xB200)	No	Yes (TP=8)	No	Yes (FSDP for opt state)
70B pretrain, multi-node	Yes (across nodes)	Yes (within node)	Optional	Yes
GPT-4-class frontier model	Yes	Yes	Yes	Yes (all 4 combined)

Pick the Parallelism — Worked Scenarios

Q: Model fits on one GPU; you want 4x more throughput.

A: Data parallel (DDP). Simplest, default choice — you're not memory constrained.

Q: Model is 200B params; the biggest single layer fits on one GPU.

A: FSDP / ZeRO-3. Memory is the constraint, not any one layer — shard weights+grads+opt state.

Q: One single transformer layer has weights that don't fit on one GPU.

A: Tensor parallelism. You must split the weight matrix itself, not just the batch.

Q: You want data-parallel speed but also need to shard optimizer state.

A: FSDP or ZeRO-2 — gives DDP-like training loop with memory-efficient optimizer sharding.

Session 1 Summary

- Four parallelism axes: data (default), tensor (one layer too big), pipeline (model too deep), sharded (memory-bound)
- 3D parallelism combines all three at frontier scale — each GPU gets a (p, t, d) coordinate
- Vocabulary: world size, rank, local rank, node rank, process group — torchrun sets these for you
- NCCL is the default GPU communication backend; Gloo for CPU debugging, MPI for legacy HPC
- All-reduce is the workhorse primitive — it decomposes into reduce-scatter + all-gather
- Rule of thumb: start with DDP, move to FSDP on OOM, only reach for tensor/pipeline/3D at frontier scale